

JWT Attacks & API Security

How JWT is (mis)used

JSON Web Tokens = claims + signature. Server 'verifies' but trust must be earned. Common flaws:

Flaw	Demo
alg:none	<code>{"alg":"none","typ":"JWT"}</code> + payload + empty signature – accepted by naive verifiers
RS256→HS256 confusion	treat the RSA public key bytes as the HMAC secret; forge with that 'secret'
Weak secret (HS256)	hashcat -m 16500 with rockyou → forge
Missing exp/iat	tokens are forever (or long-lived)
jku/x5u poisoning	point 'jku' to your JWKS → server fetches YOU
kid injection / path traversal	kid ⇒ path to file → server uses YOUR file
Replay	no nonce/jti/one-time semantics

Verify-before-trust checklist

Validate signature with the configured public key (from trusted JWKS)

Hard-require alg (deny 'none'); never accept key from the token

Check exp/iat/aud/iss claims

Rotate signing keys; revoke on logout (real revocation = hard; short exp helps)

Own the JWKS endpoint & its retrieval (allowlisted URL)

API security (the modern app surface)

BOLA / IDOR: object-level authz missing (GET /orders/{otherUserId})

BFLA: function-level authz (low-priv calling admin function)

Mass assignment: `PUT /users` with role=admin JSON

Rate limiting: enumeration, brute, scraping

Pagination abuse: deep pages bypass perf limits (DoS)

Verb confusion: alternate methods (PUT vs PATCH) performing same action with weaker checks

```
# manual API probing (Repeater):  
GET /api/users/1    -> 200 (me?)  
GET /api/users/2    -> 200 (neighbors!)  
PUT /api/users/me {"role":"admin"} -> accepted?  
POST /api/search {"q":""} -> mass enumeration?
```

Defense is boring and effective

Centralize authz: one middleware checks object ownership every access

Field-level allowlists on writes; reject unknown fields

Enforce rate limits + logging; add jti nonces for replay protection

APIs make appsec *mechanical*: authorize per object, verify per signature, rate-limit per identity.

[← Insecure Deserialization & X...](#)

[Business Logic Flaws & AuthZ G...](#)